

Online Restaurant Ordering System API

Technical Document

Team: Team Delta

Date: July 31, 2026

For developers maintaining or integrating with this API: architecture, endpoints, code patterns, and local setup.

1. Architecture Overview

The Online Restaurant Ordering System (ROS) API is a FastAPI + SQLAlchemy backend. HTTP requests hit router modules, routers call controllers, and controllers talk to the database through SQLAlchemy sessions. Pydantic schemas validate request bodies and shape JSON responses.

Part 2 of the project built on that pattern and turned it into a full restaurant system: 12 tables with CRUD, guest checkout, payments, promo codes, reviews, inventory alerts, menu search, revenue reports, low-performing dish reports, demo seed data, and a pytest suite.

Request path

```
Client (/docs or another HTTP client)
-> Router    (api/routers/)
-> Controller (api/controllers/)
-> Model + Session (api/models/, dependencies/database.py)
-> SQLite ros.db or MySQL
-> Schema -> JSON response
```

Project layout

Path	Role
api/main.py	App entry; loads models and registers routers
api/routers/	HTTP methods, paths, query/path params
api/controllers/	CRUD and business logic
api/models/	SQLAlchemy tables / relationships
api/schemas/	Pydantic Create / Update / response models
api/dependencies/	config.py settings + database engine/session
api/seed.py	Demo data for all tables
api/tests/	pytest against in-memory SQLite

Tables (Part 2)

Table	Notes
customers	Customer records; guest orders may use null customerID
restaurant_employees	Staff linked to orders
restaurant_managers	Managers for menu, inventory, promos, reports
inventory	Ingredient stock + minimum_quantity
menu_items	Menu dishes, price, dietary_type, is_available
menu_item_inventory	Composite PK: item_id + ingredient_id
promo_codes	String PK; discount, expirationDate, active
orders	Status, type, total, estimatedTime, optional FKs
order_details	Line items: order_id, item_id, quantity
payments	Method, status, amount; unique orderID
reviews	Rating 1-5 + comment on a menu item
reports	Named report rows created by managers

Business logic added in Part 2

- GET /inventory/alerts - stock at or below minimum
- POST /menuitems/recompute-availability (+ hook on inventory PUT)
- GET /orders/?start_date=&end_date= - date filter
- GET /menuitems/?dietary_type=&q= - filter/search
- GET /reports/revenue/daily and /reports/revenue/trends
- POST /promocodes/apply - validate promo and discount total
- GET /reports/low-performing - low ratings / low order volume

2. Endpoint Documentation

Base URL: `http://127.0.0.1:8000`. JSON bodies use `Content-Type: application/json`. Successful calls typically return 200; unknown IDs return 404.

2.1 CRUD resources

Each resource supports POST /, GET /, GET /{id}, PUT /{id}, DELETE /{id}.

Resource	Prefix	ID	Create body (main fields)
Customers	/customers	item_id	name, email, phone?, hasAccount
Employees	/employees	item_id	employee_id, name, role
Managers	/managers	item_id	name, email
Inventory	/inventory	item_id	ingredient_name, quantity, minimum_quantity
Menu items	/menuitems	item_id	item_name, description?, price, category?, dietary_type?, is_available
Menu-item inventory	/menuiteminventory	item_id/ingredient_id	item_id, ingredient_id, quantity_required
Orders	/orders	item_id	orderStatus, orderType, totalPrice, estimatedTime, promoCode?, customerID?, employeeID?
Order details	/orderdetails	item_id	order_id, item_id, quantity
Payments	/payments	item_id	orderId, paymentMethod, paymentStatus, amount
Promo codes	/promocodes	promo_code	promoCode, discountAmount, expirationDate, active, managerID
Reports	/reports	item_id	report_name, generated_by_manager_id?
Reviews	/reviews	item_id	rating (1-5), comment?, customerID, item_id

2.2 Sample requests and responses

Orders

```
POST /orders/

{
  "orderStatus": "pending",
  "orderType": "takeout",
  "totalPrice": "9.50",
  "estimatedTime": 15,
  "promoCode": null,
  "customerID": null,
  "employeeID": 1
}

# 200 response
{
  "orderId": 7,
  "orderDate": "2026-07-31T20:00:00",
  "orderStatus": "pending",
  "orderType": "takeout",
  "totalPrice": "9.50",
  "estimatedTime": 15,
  "promoCode": null,
  "customerID": null,
  "employeeID": 1
}

GET /orders/
GET /orders/?start_date=2026-07-01&end_date=2026-07-31
GET /orders/7
PUT /orders/7 {"orderStatus": "completed"}
DELETE /orders/7
```

Optional query params on list: `start_date`, `end_date` (YYYY-MM-DD). `customerID` may be null for guest checkout.

Order details

```
POST /orderdetails/

{
  "order_id": 7,
  "item_id": 1,
  "quantity": 2
}

# 200 response
{
  "id": 8,
  "order_id": 7,
  "item_id": 1,
  "quantity": 2
}
```

Payments

```
POST /payments/

{
  "orderId": 7,
  "paymentMethod": "card",
  "paymentStatus": "paid",
  "amount": "9.50"
}

# 200 response
{
  "paymentID": 7,
  "orderId": 7,
  "paymentMethod": "card",
  "paymentStatus": "paid",
  "amount": "9.50"
}
```

Menu items (including search)

```
POST /menuitems/

{
  "item_name": "Club Sandwich",
  "description": "Turkey and cheddar",
  "price": "9.50",
  "category": "Sandwiches",
  "dietary_type": "none",
  "is_available": true
}

GET /menuitems/
GET /menuitems/?dietary_type=vegan&q=soup
GET /menuitems/1
PUT /menuitems/1
DELETE /menuitems/1
POST /menuitems/recompute-availability
```

List query params: dietary_type (optional), q (optional keyword).

Inventory + alerts

```
POST /inventory/

{
  "ingredient_name": "Tomato",
  "quantity": 3,
  "minimum_quantity": 5
}

GET /inventory/
GET /inventory/1
PUT /inventory/1
DELETE /inventory/1
GET /inventory/alerts
```

/alerts returns rows where quantity <= minimum_quantity. PUT also triggers menu availability recompute.

Promo codes + apply

```
POST /promocodes/

{
  "promoCode": "WELCOME10",
  "discountAmount": "10.00",
  "expirationDate": "2026-12-31T23:59:59",
  "active": true,
  "managerID": 1
}

GET /promocodes/WELCOME10
PUT /promocodes/WELCOME10
DELETE /promocodes/WELCOME10

POST /promocodes/apply

{
  "promoCode": "WELCOME10",
  "orderID": 7
}

# 200 response
{
  "promoCode": "WELCOME10",
  "orderID": 7,
  "originalTotal": "19.50",
  "discountAmount": "10.00",
  "newTotal": "9.50"
}
```

Reviews

```
POST /reviews/

{
  "rating": 5,
  "comment": "Really good sandwich",
  "customerID": 1,
  "item_id": 1
}

GET /reviews/
GET /reviews/1
```

```
PUT /reviews/1
DELETE /reviews/1
```

Customers, employees, managers

```
POST /customers/
{
  "name": "Taylor Morgan",
  "email": "taylor@example.com",
  "phone": "704-555-0101",
  "hasAccount": false
}

POST /employees/
{
  "employee_id": "E001",
  "name": "Sam Patel",
  "role": "cashier"
}

POST /managers/
{
  "name": "Alex Rivera",
  "email": "alex@ros.demo"
}
```

Each also has GET /, GET /{id}, PUT /{id}, DELETE /{id}.

Menu-item inventory + reports

```
POST /menuiteminventory/
{
  "item_id": 1,
  "ingredient_id": 4,
  "quantity_required": 1
}
GET /menuiteminventory/1/4
PUT /menuiteminventory/1/4
DELETE /menuiteminventory/1/4

POST /reports/
{
  "report_name": "Weekly Sales",
  "generated_by_manager_id": 1
}

GET /reports/revenue/daily?date=2026-07-31
# {"date":"2026-07-31","total_revenue":"4.25","payment_count":1}

GET /reports/revenue/trends?start_date=2026-07-01&end_date=2026-07-31
GET /reports/low-performing
GET /reports/low-performing?max_avg_rating=2.5&max_order_count=2
```

Revenue totals only include payments with paymentStatus = paid, grouped by orderDate.

3. Code Examples

3.1 Controller create (orders)

```
api/controllers/orders.py

def create(db: Session, request):
    new_item = model.Order(
        orderStatus=request.orderStatus,
        orderType=request.orderType,
        totalPrice=request.totalPrice,
        estimatedTime=request.estimatedTime,
        promoCode=request.promoCode,
        customerID=request.customerID,
        employeeID=request.employeeID,
    )
    try:
        db.add(new_item)
        db.commit()
        db.refresh(new_item)
    except SQLAlchemyError as e:
        error = str(e.__dict__['orig'])
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=error,
        )
    return new_item
```

Builds an Order row from the request schema, commits it, refreshes so generated fields like orderID are loaded, then returns that object for the response model.

3.2 Router wiring

```
api/routers/orders.py

router = APIRouter(tags=['Orders'], prefix='/orders')

@router.post('/', response_model=schema.Order)
def create(request: schema.OrderCreate, db: Session = Depends(get_db)):
    return controller.create(db=db, request=request)

@router.get('/', response_model=list[schema.Order])
def read_all(
    start_date: Optional[date] = None,
    end_date: Optional[date] = None,
    db: Session = Depends(get_db),
):
    return controller.read_all(
        db, start_date=start_date, end_date=end_date
    )

@router.get('/{item_id}', response_model=schema.Order)
def read_one(item_id: int, db: Session = Depends(get_db)):
    return controller.read_one(db, item_id=item_id)

@router.put('/{item_id}', response_model=schema.Order)
def update(
    item_id: int,
    request: schema.OrderUpdate,
    db: Session = Depends(get_db),
):
    return controller.update(db=db, request=request, item_id=item_id)

@router.delete('/{item_id}')
```

```
def delete(item_id: int, db: Session = Depends(get_db)):
    return controller.delete(db=db, item_id=item_id)
```

The router owns HTTP concerns only. `Depends(get_db)` injects a per-request session. Other resources follow this same create / read_all / read_one / update / delete layout.

3.3 DB session + router registration

```
Session + registration
# api/dependencies/database.py
SessionLocal = sessionmaker(
    autocommit=False, autoflush=False, bind=engine
)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# api/routers/index.py (called from main.py)
def load_routes(app):
    app.include_router(orders.router)
    app.include_router(order_details.router)
    app.include_router(customers.router)
    # ... remaining resources ...
    app.include_router(reviews.router)
```

`get_db()` opens a session for the request and closes it afterward. `load_routes()` mounts every `APIRouter` onto the FastAPI app.

4. Development Environment

Required tools

- Python 3.10+
- pip, Git
- Browser for <http://127.0.0.1:8000/docs>
- MySQL only if SQLite is disabled in config

Dependencies

```
pip install -r requirements.txt
```

Frameworks/libraries: FastAPI, Uvicorn, SQLAlchemy, PyMySQL, Pydantic, email-validator, pytest, httpx, cryptography.

Database

```
# api/dependencies/config.py
use_sqlite = True
sqlite_path = "./ros.db"

# MySQL:
use_sqlite = False
# set db_host, db_name, db_user, db_password, db_port
```

Tables are created on app startup via `api/models/model_loader.py`.

Run the API

```
uvicorn api.main:app --reload

# Interactive docs: http://127.0.0.1:8000/docs
```



```
python -m api.seed  
python -m api.seed --force  
pytest api/tests
```